

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG**



**BÁO CÁO THỰC HÀNH NỀN TẢNG CÔNG
NGHỆ SỐ**

**XÂY DỰNG MINIGAME BẰNG NGÔN
NGŨ C
BUILD A MINIGAME USING
C PROGRAMMING**

**Sinh viên: Lưu Vinh Hiển
Mã số: B2509828
Khóa: K51**

**Sinh viên: Bùi Nguyên Thảo
Mã số: B2509854
Khóa: K51**

Cần Thơ, 03/2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN**



**XÂY DỰNG MINIGAME BẰNG NGÔN NGỮ C
BUILD A MINIGAME USING
C PROGRAMMING**

**Người hướng dẫn
TS. Bùi Võ Quốc Bảo**

**Sinh viên thực hiện
Luu Vinh Hiễn
Mã số: B2509828
Khóa: K51**

**Sinh viên thực hiện
Bùi Nguyên Thảo
Mã số: B2509854
Khóa: K51**

Cần Thơ, 03/2026

LỜI CẢM ƠN

Nhóm chúng em xin chân thành cảm ơn thầy Bùi Võ Quốc Bảo đã tạo điều kiện để chúng em thực hiện và hoàn thành dự án này.

Trong quá trình thực hiện, nhóm đã có cơ hội vận dụng những kiến thức đã học vào thực tế, đồng thời rèn luyện kỹ năng làm việc nhóm và giải quyết vấn đề.

Nhóm chúng em cũng xin cảm ơn sự phối hợp và nỗ lực của các thành viên trong nhóm đã cùng nhau hoàn thành dự án đúng tiến độ.

Do kiến thức và kinh nghiệm còn hạn chế, dự án khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những góp ý để có thể hoàn thiện hơn trong tương lai.

Nhóm chúng em xin chân thành cảm ơn!

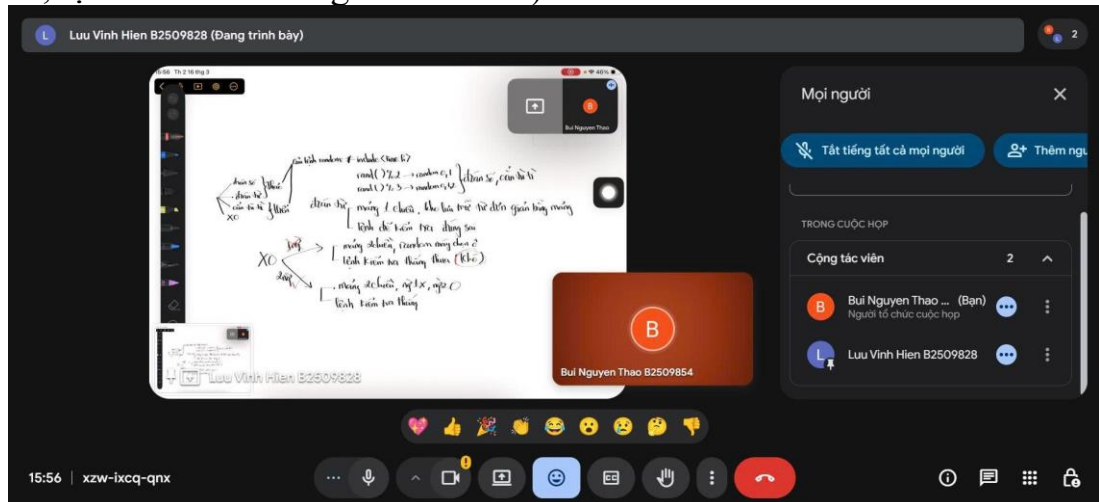
MỤC LỤC

1. Ý tưởng ứng dụng:	1
2. Nghiên cứu và bắt đầu viết mã:	1
3. Các tính năng sau những lần phát triển.....	2
• Đoán số:	2
• Đoán từ:	2
• Oẳn tù tì:	2
• XO (3x3):.....	2
4. Sản phẩm hoàn thiện:	3
• Giao diện ban đầu:	3
• Đoán số:	3
• Đoán chữ:.....	3
• Oẳn tù tì:	3
• XO (3x3):.....	3
• Thoát game:	3
5. Quy trình phát triển game:	3
• Đoán số:	3
• Đoán từ:	5
• Oẳn tù tì:	7
• XO (3x3):.....	9
6. Link dẫn đến kho lưu trữ code trên GitHub.....	11
7. Link dẫn đến Youtube.....	11
8. Phản ánh quá trình thực hiện dự án.....	11

TUẦN 1

1. Ý tưởng ứng dụng:

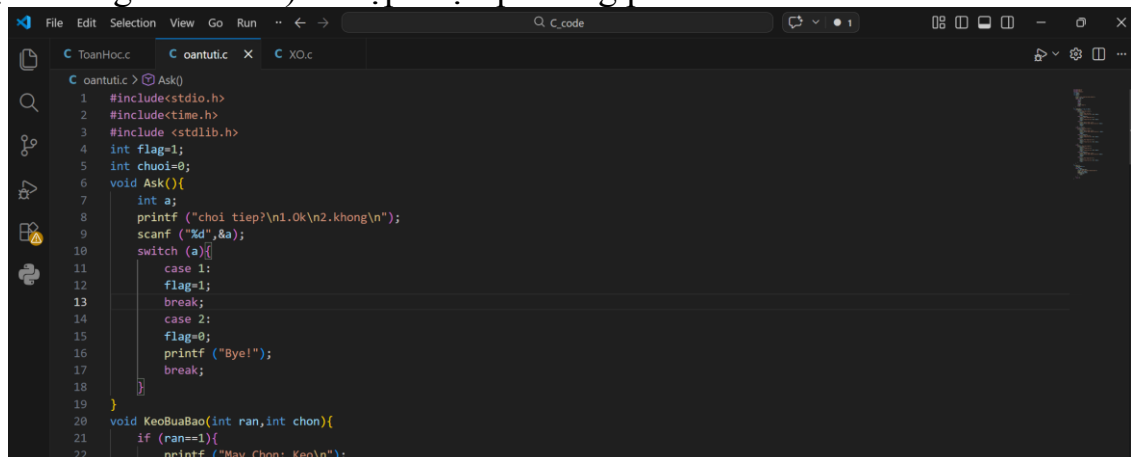
Tự lên ý tưởng: ý tưởng là code một game có nhiều mini game trong đó, game 1: đoán số (có chế độ dễ, thường, khó, có chuỗi thắng, dùng lệnh random), game 2: đoán từ (có tính số lần đoán trúng, dùng lệnh random, kiến thức mảng và string), game 3: oẳn tù tì (dùng lệnh random, và có tính chuỗi thắng), game 4: XO (game 2 người chơi, dùng nhiều kiến thức về mảng 2 chiều, lệnh kiểm tra thắng thua và huê).



2. Nghiên cứu và bắt đầu viết mã:

Sử dụng các kiến thức lập trình như: if-else, vòng lặp, mảng 1 chiều và 2 chiều, chuỗi dữ liệu, function và tự nghiên cứu và tìm hiểu câu lệnh về random để lập trình game.

Viết mã code đầu tiên và đưa lên Github. Sửa lỗi(sử dụng ChatGPT để tiết kiệm thời gian tìm lỗi) và cập nhật qua từng phiên bản.



```

1 // DOAN SO
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5 int main () {
6     int guess;
7     // random so
8     srand(time(NULL));
9     int secret = rand() % 101;
10
11     // nguoi choi doan
12     scanf("%d", &guess);
13
14     if (guess == secret) {
15         printf("Ban da doan dung");
16     }
17     else if (guess > secret) {
18         printf("So can doan nho hon");
19     }
20     else {
21         printf("So can doan lon hon");
22     }
23
24     return 0;
25 }

```

3. Các tính năng sau những lần phát triển

- **Đoán số:**

Code lần đầu: 1 chế độ chơi, không có ghi lại chuỗi thắng.

Những lần sửa code tiếp theo đã thêm độ khó cho trò chơi, đã thêm chuỗi thắng qua nhiều lần hội ý.

- **Đoán từ:**

Code lần 1: thư viện từ để đoán có 10 từ.

Những lần sửa code tiếp theo đã thêm nhiều từ vào thư viện từ hơn, có thêm nhiều chủ đề mới như màu sắc, động vật và trái cây.

- **Ẩn từ từ:**

Code lần 1: chưa có chuỗi.

Những lần sửa code tiếp theo đã thêm tính năng chuỗi vào.

- **XO (3x3):**

Code lần 1: chưa có chuỗi thắng, chưa có hàm để kiểm tra huê.

Những lần sửa code tiếp theo đã thêm ghi lại chuỗi thắng, và hàm kiểm tra huê.

TUẦN 2

4. Sản phẩm hoàn thiện:

- **Giao diện ban đầu:**

- **Đoán số:**

Có nhiều chế độ chơi khác nhau, chế độ dễ là đoán các số random trong khoảng 0-50, chế độ bình thường là từ 0-100, và chế độ khó là từ 0-500. Khi đoán số, nếu số người chơi đoán nhỏ hơn số máy random sẽ được nhận thông báo nhỏ hơn và ngược lại, mỗi lần đoán đều được ghi lại.

- **Đoán chữ:**

Máy lưu rất nhiều từ để đoán với nhiều chủ đề hay như màu sắc, động vật, và trái cây. Khi đoán đúng sẽ hiện chữ đúng, chữ có trong cụm trên thì sẽ được đưa đến đúng chỗ. Có gợi ý và dấu gạch dưới để chỉ số từ cần ghi.

- **Oẳn tù tì:**

Dùng lệnh random để random một trong ba bộ kéo, búa, bao. Hàm kiểm tra thắng thua và ghi lại số lần thắng. Cuối cùng là in chuỗi thắng khi thua.

- **XO (3x3):**

Có các hàm để xác định thắng thua (đường ngang và đường chéo) và hàm kiểm tra huê, khi thắng thì chuỗi thắng tăng 1, huê thì không tăng.

- **Thoát game:**

5. Quy trình phát triển game:

- **Đoán số:**

- **Giới hạn kiến thức xoay quanh:**

Sử dụng cấu trúc điều kiện (if-else)

Vòng lặp (while)

Sinh số ngẫu nhiên

Nhập/xuất dữ liệu

Tổ chức logic chương trình

- **Phân tích yêu cầu:**

- + Hiển thị giao diện trò chơi

- + Cho phép người chơi chọn độ khó

- + Sinh số bí mật ngẫu nhiên theo độ khó

- + Cho phép người chơi nhập số dự đoán

- + So sánh và đưa ra gợi ý (lớn hơn / nhỏ hơn)

- + Đếm số lần đoán

- + Cho phép chơi lại

- **Thiết kế thuật toán:**
 - + **Bước 1: Khởi tạo**
Khai báo các biến:
 - guess: số người chơi nhập
 - secret: số bí mật
 - streak: số lần đoán
 - level: độ khó
 - play: lựa chọn chơi lại
 - + **Bước 2: Vòng lặp trò chơi**
Sử dụng vòng lặp while để cho phép người chơi chơi lại nhiều lần
 - + **Bước 3: Chọn độ khó**
Hiển thị menu:
 - 1: 0 → 50
 - 2: 0 → 100
 - 3: 0 → 500
Kiểm tra nhập liệu, nếu nhập sai thì cho người chơi nhập lại.
 - + **Bước 4: Chọn số ngẫu nhiên**
Sử dụng:
 - srand(time(NULL)) để tạo seed
 - rand() để sinh số
Tùy theo độ khó:
 - level 1 → rand() % 51
 - level 2 → rand() % 101
 - level 3 → rand() % 501
 - + **Bước 5: Người chơi đoán**
Nhập số lần đầu
Tăng biến đếm streak
 - + **Bước 6: So sánh kết quả**
Nếu:
 - guess > secret → thông báo “NHỎ HƠN”
 - guess < secret → thông báo “LỚN HƠN”
Tiếp tục yêu cầu nhập lại cho đến khi đúng.
 - + **Bước 7: kết thúc lượt chơi**
Thông báo đoán đúng
Hiển thị số lần đoán
 - + **Bước 8: chơi lại**
Hỏi người chơi có muốn chơi tiếp không

Nếu 'y' → quay lại đầu

Nếu 'n' → kết thúc chương trình

– **Cài đặt chương trình:**

Chương trình được viết bằng ngôn ngữ C, sử dụng các thư viện:

- stdio.h: nhập/xuất dữ liệu
- stdlib.h: hàm rand()
- time.h: tạo seed ngẫu nhiên

Các cấu trúc chính:

- Vòng lặp while
- Câu lệnh điều kiện if-else
- Hàm sinh số ngẫu nhiên

– **Kiểm thử chương trình:**

Trường hợp 1:

- Nhập đúng ngay lần đầu → chương trình kết thúc nhanh

Trường hợp 2:

- Nhập sai nhiều lần → chương trình vẫn hoạt động đúng

Trường hợp 3:

- Nhập sai độ khó → yêu cầu nhập lại

• **Đoán từ:**

– **Giới hạn kiến thức xoay quanh:**

- Xử lý chuỗi (string)

Sử dụng mảng ký tự

Vòng lặp và điều kiện

So sánh chuỗi

Sinh dữ liệu ngẫu nhiên

– **Phân tích yêu cầu:**

Chương trình cần thực hiện các chức năng sau:

- Hiển thị tiêu đề trò chơi
- Chọn ngẫu nhiên một nhóm từ (trái cây, động vật, màu sắc)
- Chọn ngẫu nhiên một từ trong nhóm
- Hiển thị gợi ý cho người chơi
- Ẩn từ bằng dấu "_"
- Cho phép người chơi đoán từng ký tự
- Cập nhật trạng thái từ sau mỗi lần đoán
- Kiểm tra khi đoán hoàn thành
- Cho phép chơi lại

– **Thiết kế thuật toán:**

+ **Bước 1: Khởi tạo**

Khai báo biến:

- group: nhóm từ

- tu: vị trí từ trong mảng
- secret: từ bí mật
- display: trạng thái hiển thị
- doan: ký tự người chơi nhập
- play: lựa chọn chơi lại
- + **Bước 2: Vòng lặp trò chơi**
Sử dụng vòng lặp while để cho phép chơi nhiều lần.
- + **Bước 3: Chọn dữ liệu ngẫu nhiên**
Sử dụng srand(time(NULL)) để tạo seed
Chọn nhóm:
 - group = rand() % 3
- + **Bước 4: chọn từ bí mật**
Dựa vào nhóm:
 - Nhóm 0: Trái cây
 - Nhóm 1: Động vật
 - Nhóm 2: Màu sắc
 Sau đó chọn từ:
 - rand() % số lượng phần tử trong mảng
- + **Bước 5: trạng thái hiển thị**
Lấy độ dài từ bằng strlen()
Tạo mảng display chứa ký tự '_'
In ra dạng: _ _ _ _
- + **Bước 6: vòng lặp đoán từ**
Lặp cho đến khi:
 - display giống hoàn toàn secret
- + **Bước 7: cập nhật trạng thái**
Hiển thị từ hiện tại sau mỗi lần đoán
Ví dụ: a _ p _ e
- + **Bước 8: Kết thúc trò chơi**
Khi display == secret:
 - Thông báo chiến thắng
 - Hiển thị từ đúng
- + **Bước 9: Chơi lại**
Hỏi người chơi:
 - 'y' → chơi tiếp
 - 'n' → kết thúc
- **Cài đặt chương trình:**
Chương trình sử dụng các thư viện:
 - stdio.h: nhập/xuất
 - stdlib.h: rand()

- time.h: seed ngẫu nhiên
- string.h: xử lý chuỗi

Các kỹ thuật chính:

- Mảng 2 chiều lưu danh sách từ
- Con trỏ ký tự (char*)
- Hàm strcmp để so sánh chuỗi
- Hàm strlen để lấy độ dài

– **Kiểm thử chương trình:**

- Trường hợp 1:
 - Người chơi đoán đúng toàn bộ → chương trình kết thúc đúng
- Trường hợp 2:
 - Người chơi đoán sai nhiều lần → chương trình vẫn hoạt động
- Trường hợp 3:
 - Đoán trùng ký tự → vẫn cập nhật đúng
- **Oán tù tù:**

– **Giới hạn kiến thức xoay quanh:**

Sử dụng hàm (function)

Biến toàn cục

Cấu trúc điều kiện

Sinh số ngẫu nhiên

Tổ chức chương trình thành nhiều phần

– **Phân tích yêu cầu:**

Chương trình cần thực hiện:

- Cho người chơi chọn Kéo, Búa hoặc Bao
- Máy tính chọn ngẫu nhiên
- So sánh kết quả và thông báo thắng/thua/hòa
- Tính chuỗi thắng liên tiếp
- Kết thúc khi thua
- Cho phép chơi lại

– **Thiết kế thuật toán:**

+ **Bước 1: Khởi tạo**

Biến toàn cục:

- flag: điều khiển vòng lặp trò chơi
- chuoi: số lần thắng liên tiếp

Biến cục bộ:

- chon: lựa chọn của người chơi
- ran: lựa chọn của máy

+ **Bước 2: Vòng lặp trò chơi**

Sử dụng while(flag) để lặp cho đến khi người chơi dừng.

+ **Bước 3: lựa chọn của máy**

Sử dụng:

- `rand() % 3 + 1`
- Tương ứng:
 - 1 → Kéo
 - 2 → Búa
 - 3 → Bao
- + **Bước 4: nhập lựa chọn của người chơi**
 Hiển thị menu:
 - 1: Kéo
 - 2: Búa
 - 3: Bao
- + **Bước 5: Xử lý kết quả**
 Hàm nhận 2 tham số:
 - `ran`: lựa chọn máy
 - `chon`: lựa chọn người chơi
 Các trường hợp:
 - Hòa → không thay đổi chuỗi
 - Thắng → tăng chuỗi (`chuoi++`)
 - Thua → reset chuỗi về 0 và kết thúc lượt chơi
- + **Bước 6: xử lý sau khi thua**
 Khi người chơi thua:
 - `flag = 0`
 - Gọi hàm kết thúc
- + **Bước 7: hàm kết thúc**
 Hiển thị:
 - 1: chơi tiếp
 - 2: thoát
 Nếu chọn 1 → `flag = 1`
 Nếu chọn 2 → `flag = 0` và kết thúc
- **Cài đặt chương trình**
 Chương trình sử dụng các thư viện:
 - `stdio.h`: nhập/xuất
 - `stdlib.h`: `rand()`
 - `time.h`: seed ngẫu nhiên
 Các thành phần chính:
 - Hàm `main()`: điều khiển chương trình
 - Hàm `KeoBuaBao()`: xử lý logic thắng thua
 - Hàm `Ask()`: xử lý chơi lại
- **Kiểm thử chương trình**
 - Trường hợp 1:
 - Người chơi thắng liên tiếp → chuỗi tăng đúng
 - Trường hợp 2:

- Người chơi thua → chuỗi reset về 0
 - Trường hợp 3:
 - Chọn chơi lại → chương trình tiếp tục
 - **XO (3x3):**
- **Giới hạn kiến thức xoay quanh:**
 Sử dụng mảng 2 chiều
 Xây dựng hàm kiểm tra điều kiện
 Kiểm tra logic thắng/thua
 Xử lý vòng lặp và nhập dữ liệu
- **Phân tích yêu cầu:**
 Tạo bảng 3x3 ban đầu
 Cho 2 người chơi nhập tọa độ
 Kiểm tra nước đi hợp lệ
 Cập nhật bảng sau mỗi lượt
 Kiểm tra thắng
 Kiểm tra hòa
 Kết thúc trò chơi khi có kết quả
- **Thiết kế thuật toán:**
- + **Bước 1: Khởi tạo**
 Tạo mảng 2 chiều:
 - `char a[3][3]`
 Gán tất cả phần tử bằng ' ' (ô trống)
 - + **Bước 2: Hiển thị bảng**
 Duyệt mảng 2 chiều
 In ra dạng:


```

x | o |
---|---|---
  | x |
---|---|---
  |  | o
    
```
 - + **Bước 3: Hàm kiểm tra thắng**
 Kiểm tra:
 - 3 hàng ngang
 - 3 cột dọc
 - 2 đường chéo
 Điều kiện:
 - 3 ô giống nhau và khác khoảng trắng
 Nếu thỏa → trả về 1
 - + **Bước 4: kiểm tra hòa**
 Duyệt toàn bộ bảng

- Nếu còn ô trống → chưa hòa
- Nếu không còn và chưa thắng → hòa
- + **Bước 5: Người chơi đoán**
Sử dụng vòng lặp while(1) để chạy liên tục
- + **Bước 6: người chơi 1**
 - Nhập hàng và cột
 - Kiểm tra:
 - Có nằm trong [0,2]
 - Ô có trống không
 - Nếu hợp lệ:
 - Gán 'x'
 - In bảng
- + **Bước 7: người chơi 2**
Thực hiện tương tự
Gán 'o'
- + **Bước 8: chơi lại**
Nếu có người thắng → in "WIN"
Nếu hòa → in "Hoa!"
- **Cài đặt chương trình:**
Chương trình gồm các thành phần:
 - Hàm Checkwin():
Kiểm tra tất cả điều kiện thắng
 - Hàm Checktie():
Kiểm tra trạng thái hòa
 - Hàm inbang():
Hiển thị bảng trò chơi
 - Hàm main():
Điều khiển toàn bộ chương trình
- **Kiểm thử chương trình:**
Người chơi thắng theo hàng → đúng
 - Trường hợp 2:
Người chơi thắng theo cột → đúng
 - Trường hợp 3:
Người chơi thắng theo đường chéo → đúng
 - Trường hợp 4:
Bảng đầy → hòa
 - Trường hợp 5:
Nhập sai tọa độ → yêu cầu nhập lại

TUẦN 3

6. *Link dẫn đến kho lưu trữ code trên GitHub*

<https://github.com/thaob2509854-bit/minigame.git>

7. *Link dẫn đến Youtube*

<https://youtu.be/qG8EupooH2o?si=5vTLd6zAxx475Oww>

8. *Phản ánh quá trình thực hiện dự án*

- **Khó khăn gặp phải:**

- Gặp lỗi trong quá trình lập trình như sai logic chương trình, lỗi nhập/xuất dữ liệu (scanf), vòng lặp hoạt động chưa đúng.
- Khó khăn khi kết hợp các chức năng nhỏ (menu, đoán số, đoán chữ,...) thành một chương trình hoàn chỉnh.
- Một số lỗi phát sinh khi chạy chương trình nhưng khó xác định nguyên nhân.
- Cách giải quyết và sử dụng AI:
 - Nhóm chủ động tìm hiểu tài liệu, thử nhiều cách khác nhau và kiểm tra từng phần của chương trình để sửa lỗi.
 - Có sử dụng AI như ChatGPT để gợi ý hướng giải quyết, giải thích lỗi và tối ưu code.
 - Tuy nhiên, nhóm luôn đọc hiểu, kiểm tra lại và chỉnh sửa cho phù hợp, không sao chép trực tiếp.

- **Những kỹ năng và hiểu biết đạt được:**

- Nâng cao kỹ năng lập trình C: sử dụng biến, điều kiện, vòng lặp, hàm và xây dựng cấu trúc chương trình.
- Phát triển tư duy logic và kỹ năng debug khi xử lý lỗi.
- Cải thiện kỹ năng làm việc nhóm: phân chia công việc, phối hợp và hỗ trợ lẫn nhau.
- Biết cách sử dụng AI như một công cụ hỗ trợ học tập hiệu quả.
- Tăng khả năng tự học và áp dụng kiến thức vào thực tế.
- Kỹ năng thao tác trên Word, PowerPoint đã được áp dụng vào bài.
- Kỹ năng quay màn hình và chỉnh sửa video.

- **Kết luận:**

Dự án giúp nhóm củng cố kiến thức đã học, rèn luyện kỹ năng thực hành và tích lũy thêm kinh nghiệm trong quá trình lập trình.